ChaiDocs

Jinja Templates App in Django

Jinja2 is a template engine for Python. It is used in Django to render templates. It is a very powerful template engine that can be used to render HTML, XML, and other formats. It is also used to render templates for the Django admin interface.

Installation

If you are in Django, you don't need to install Jinja2 separately. It is already installed with Django. Django also comes with a built-in template configurations that allows you to use Jinja2 templates.

Jinja2 templates are written in a simple text format called HTML. The syntax is very similar to HTML, but with some additional features. You need to inject variables into the template using the `{{ variable }}` syntax. For example, if you want to display a name, you can use the following code:

```
Hello {{ name }}!
```

This will display the name of the person who is currently logged in, if there is one.

Common Template Tags

Following are some of the common template tags that you can use in Jinja2 templates:

% if %

The `{% if %}` tag is used to conditionally display content in a template. It takes a boolean expression as an argument, and if the expression evaluates to `True`, the content inside the tag will be displayed. If the expression evaluates to `False`, the content will be skipped.

For example, the following code will display a greeting message only if the `name` variable is not empty:

```
{% if name %}
  Hello, {{ name }}!
{% endif %}
```

{% for %}

The `{% for %}` tag is used to iterate over a sequence of items. It takes a variable name and a sequence as arguments, and displays the content inside the tag for each item in the sequence.

For example, the following code will display a list of names:

```
{% for name in names %}
  {{ name }} is a name.
{% endfor %}
```

{% block %}

The `{% block %}` tag is used to define a block of content that can be overridden in child templates. It takes a name as an argument, and defines a block with that name that can be overridden in child templates.

For example, the following code defines a base template that includes a header and a footer:

```
<!DOCTYPE html>
<html>
  <head>
    <title>{% block title %}My Website{% endblock %}</title>
  </head>
  <body>
    {% block content %}
      <h1>Welcome to my website!</h1>
    {% endblock %}
    <footer>
      {% block footer %}
        <p>Copyright © 2021</p>
      {% endblock %}
    </footer>
  </body>
</html>
```

And the following code defines a child template that overrides the `content` block:

```
{% extends "base.html" %}

{% block title %}My Website{% endblock %}

{% block content %}
  <h1>Welcome to my website!</h1>
  <p>This is a child template.</p>
{% endblock %}
```

In this example, the `content` block in the child template overrides the `content` block in the base template, and the `title` block is not overridden.

{% include %}

The {% include %} tag is used to include the contents of another template file. It takes a template name as an argument, and includes the contents of the template file in the current template.

For example, the following code includes the contents of a template file called header.html:

```
{% include "header.html" %}
```

This will include the contents of the header.html template file in the current template.

{% extends %}

The {% extends %} tag is used to extend a base template. It takes a template name as an argument, and extends the base template with the contents of the template file.

For example, the following code extends the base.html template with the contents of a template file called child.html:

```
{% extends "base.html" %}

{% block content %}
 <h1>Welcome to my website!</h1>
 <p>This is a child template.</p>
{% endblock %}
```

In this example, the content block in the child template overrides the content block in the base template, and the title block is not overridden.

{% load %}

The {% load %} tag is used to load a template tag library. It takes a library name as an argument, and loads the template tag library with that name.

For example, the following code loads the static template tag library:

```
{% load static %}
```

This will load the static template tag library, which provides a set of template tags for working with static files.

{% static %}

The {% static %} tag is used to include a static file in a template. It takes a file path as an argument, and includes the contents of the file in the current template.

For example, the following code includes the contents of a CSS file called style.css:

```
<link rel="stylesheet" href="{% static 'style.css' %}">
```

This will include the contents of the style.css file in the current template.

{% url %}

The {% url %} tag is used to generate a URL for a view. It takes a view name and a set of arguments as arguments, and generates a URL for the view with those arguments.

For example, the following code generates a URL for the index view with the name argument set to 'John':

```
<a href="{% url 'index' name='John' %}">Go to the home page</a>
```

This will generate a link to the home page with the name argument set to 'John'.

Apps in Django

The most common way to organize your Django project is to use apps. An app is a self-contained module that contains models, views, templates, and other components of your project. Apps allow you to organize your code into logical units and make it easier to manage and maintain your project.

You can create it manually or use the startapp command to create a new app for you. To create an app, navigate to the directory where you want to create the app and run the following command:

```
python manage.py startapp chai
```

This will create a new directory called chai with the necessary files and directories for an app.

To add an app to your project, you need to add it to the INSTALLED_APPS setting in your project's settings.py file. You can do this by adding the app's name to the list of installed apps:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'chai',
]
```

This will add the chai app to your project's installed apps.

Templates in Apps and layout extension

In Django, templates are organized into apps. Each app can have its own templates directory, which contains the templates for that app. Create a new directory called templates in your app's directory. Inside the chai directory, create a templates directory and add a all_chai.html file to it.

Add your basic html code to the all_chai.html file.

To serve this file, we need a view and a url. Create a new file called views.py in your app's directory. Add the following code to the file:

views.py

```
from django.shortcuts import render

def all_chai(request):
    return render(request, 'all_chai.html')
```

This view will render the `all_chai.html` template when it is called.

Create a new file called `urls.py` in your app's directory. Add the following code to the file:

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.all_chai, name='all_chai'),
]
```

This urlpattern will map the root URL of the app to the `all_chai` view.

Now, we need to make aware of this new urlpattern in our project's `urls.py` file. Add the following code to the project's `urls.py` file:

urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('chai/', include('chai.urls')),
]
```

This will include the chai.urls file in the project's urlpatterns.

Now, we can access the all_chai view by going to http://localhost:8000/chai/.

Common Layout for all pages

In Django, you can create a common layout for all pages in your project by using the base.html template. Create a new file called base.html in your project's templates directory. Add the following code to the file:

base.html

```
{% load static %}

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="{% static 'style.css' %}">

  <title>
    {% block title %}
      Chai aur Django
    {% endblock title %}
  </title>
</head>
<body>
  <nav>I will add it later</nav>
  {% block content %}
  {% endblock %}
</body>
</html>
```

Now, this layout can be used for all pages in your project. To use it, you need to include it in your templates. For example, if you want to use the layout for the `all_chai` view, you can add the following code to the `all_chai.html` file:

all_chai.html

```
{% extends "base.html" %}

{% block title %}
  All Chai
{% endblock %}

{% block content %}
  <h1>All Chai</h1>
  <p>This is the all chai page.</p>
{% endblock %}
```

This will use the `base.html` layout and override the `title` and `content` blocks with the appropriate values for the `all_chai` view.

Summary

In this part, we learned about Jinja2 templates and how to use them in Django. We also learned about apps in Django and how to create a common layout for all pages in your project. By using Jinja2 templates and apps, you can create dynamic and reusable templates in Django that make your web development process more efficient and enjoyable.

Follow chai aur Django to learn more about Django and its features on youtube.

Start your journey with ChaiCode

All of our courses are available on chaicode.com. Feel free to check them out.

Complied by: Hitesh Choudhary Last updated: Apr 18, 2025

← Previous
Django Intro

Next →
Tailwind Integration

Contribute Community Sponsor